## #SKILL - 1 Git Version Control

***Tasks:***

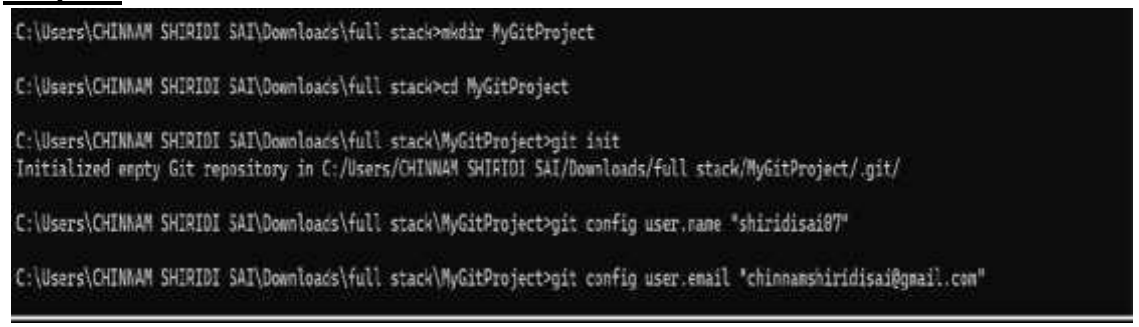
**1.** Create a new project folder, initialize Git using git init, and configure Git with your username and email.

**Commands Used:-**

mkdir MyGitProject
cd MyGitProject

git init
git config user.name "Your Name" git config
user.email "youremail@example.com"

**Output:-**




2. Create two files (example: main.java, notes.txt) with sample content, check the repository status, and add them to staging.

**Commands Used:-**

echo public class Main { public static void main(String[] args) { System.out.println("Hello Git!"); } } > main.java echo This is my first Git project. > notes.txt

git status git add main.java
notes.txt

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>echo This is my first Git project. > notes.txt

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Main.java
        notes.txt

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git add main.java notes.txt
```

3. Commit the staged files with a meaningful commit message.

**Commands Used:-**

git commit -m "Initial commit: add main.java and notes.txt"

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git commit -m "Initial commit: add main.java and notes.txt"
[master (root-commit) b8ecf2c] Initial commit: add main.java and notes.txt
 1 file changed, 1 insertion(+)
 create mode 100644 notes.txt
```

4. Create a new repository on GitHub, connect your local repo using the remote URL and push the committed changes.

**Commands Used:-** git branch -M main git remote add origin https://github.com/USERNAME/MyGitProject.git git push -u origin main

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git remote -v
origin  https://github.com/USERNAME/MyGitProject.git (fetch)
origin  https://github.com/USERNAME/MyGitProject.git (push)

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git remote remove origin

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git remote add origin https://github.com/shiridisai07/MyGitProject.git

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 266 bytes | 53.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/shiridisai07/MyGitProject.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

5. Create the first branch (example: feature-update), make modifications, and add + commit the changes.

**Commands Used:-**

git checkout -b feature-update echo Added feature update notes. >> notes.txt git add notes.txt git commit -m "Add feature update notes"

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git merge feature-update
Updating b8ecf2c..c4df3d4
Fast-forward
 notes.txt | 1 +
 1 file changed, 1 insertion(+)
```

6. Create a second branch (example: bug-fix), apply changes, and add + commit them.

**Commands Used:-** git checkout main git checkout -b bug-fix echo System.out.println("Bug fixed!"); >> main.java git add main.java git commit -m "Fix bug in main.java"

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git merge feature-update
Updating b8ecf2c..c4df3d4
Fast-forward
 notes.txt | 1 +
 1 file changed, 1 insertion(+)

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git merge bug-fix
Merge made by the 'ort' strategy.
 Main.java | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 Main.java
```

7. Switch back to the main branch and merge both branches one after the other.

**Commands Used:-** git checkout main git merge

feature-update git merge
bug-fix

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git merge bug-fix
Merge made by the 'ort' strategy.
 Main.java | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 Main.java

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git add .
```

8. If any merge conflict occurs, resolve it manually, commit the fix, and complete the merge

**Commands Used:-**

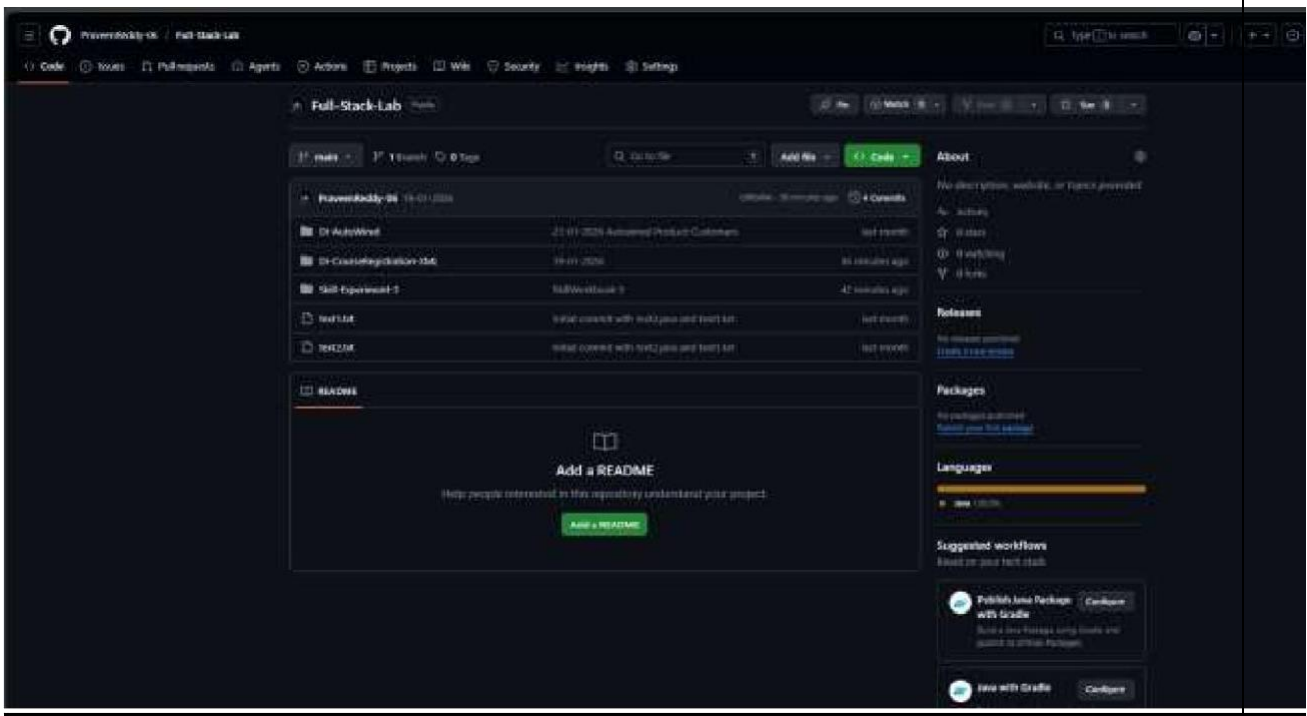
git add .
git commit -m "Resolve merge conflict"

**Output:-**

```
C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git add .

C:\Users\CHINNAM SHIRIDI SAI\Downloads\full stack\MyGitProject>git commit -m "Resolve merge conflict"
On branch main
Your branch is ahead of 'origin/main' by 3 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```




**Final GitHub Repo:-**

**Git Hub link** : https://github.com/Anil1843/FSAD_SKILL

**Conclusion:-**

In this experiment, Git was successfully used as a version control system to manage a project efficiently. The repository was initialized, user configuration was completed, and files were created, staged, and committed following the Git life cycle. Multiple branches were created to handle feature updates and bug fixes independently, ensuring safe and organized development. These branches were later merged into the main branch, demonstrating effective branch management and merge operations. Finally, the project was pushed to a remote GitHub repository, enabling centralized storage and version tracking. This exercise provided hands-on experience with essential Git commands and workflows used in real-world software development.